



UNIVERSIDAD ADOLFO IBÁÑEZ

Facultad de Ingeniería y Ciencias

Arquitectura de Computadores

Evaluación de Predicción de Saltos y Análisis de Sistemas Multiprocesador

Laboratorio 3: Branch Prediction & Multiprocessor Analysis

Profesor:

Cristián Astudillo

Ayudante:

Jorge Moreno

Fecha:
29 de abril de 2026

Índice

1. Objetivos	2
2. Introducción Teórica	2
2.1. Predicción de Saltos (Branch Prediction)	2
2.2. Sistemas Multiprocesador y Coherencia de Caché	3
2.3. Eventos de Hardware Relevantes	3
3. Requisitos y Configuración del Entorno	3
3.1. Requisitos de Hardware	3
3.2. Instalación de Herramientas	3
3.3. Verificación	4
4. Ejercicio 1: Medición del Costo de Fallos de Predicción	4
4.1. Objetivo	4
4.2. Código Base	4
4.3. Procedimiento	5
4.4. Preguntas Guía	5
5. Ejercicio 2: Análisis de Falso Compartido en Sistemas Multiprocesador	6
5.1. Objetivo	6
5.2. Código Base: Contador Compartido con Falso Compartido	6
5.3. Procedimiento	7
5.4. Preguntas Guía	7
6. Ejercicio 3: Optimización de Reducción Paralela y Coherencia	8
6.1. Objetivo	8
6.2. Código Base: Suma de un Vector	8
6.3. Procedimiento	8
6.4. Preguntas Guía	9
7. Análisis Comparativo de Técnicas	9
8. Entregables	9
8.1. Formato del Informe	9
8.2. Rúbrica de Evaluación	10

1. Objetivos

- **Objetivo General:** Evaluar el impacto de la predicción de saltos y la coherencia de caché en sistemas multiprocesador, utilizando herramientas de profiling para identificar y mitigar penalizaciones de rendimiento.
- **Objetivos Específicos:**
 - Medir cuantitativamente el costo de los fallos de predicción de saltos mediante contadores hardware.
 - Analizar cómo el patrón de ramas afecta la tasa de aciertos del predictor.
 - Identificar y resolver problemas de falso compartido (*false sharing*) en programas paralelos.
 - Utilizar `perf` para medir eventos de coherencia de caché (e.g., `cache-misses`, `LLC-load-misses`).
 - Implementar técnicas de alineación y privatización para mejorar el rendimiento en multi-núcleo.

2. Introducción Teórica

En los procesadores modernos, el rendimiento está limitado no solo por la jerarquía de memoria, sino también por la eficiencia del flujo de control y la interacción entre núcleos. Este laboratorio aborda dos grandes áreas: predicción de saltos y sistemas multiprocesador.

2.1. Predicción de Saltos (Branch Prediction)

Los procesadores segmentados intentan ejecutar instrucciones de forma especulativa antes de conocer el resultado de una rama condicional. Un fallo de predicción (*branch misprediction*) implica vaciar la segmentación y reiniciar la ejecución desde la dirección correcta, costando típicamente entre 10 y 25 ciclos (dependiendo de la longitud de la segmentación).

- **Predictor de 2 bits:** Máquina de estados que favorece ramas tomadas o no tomadas según historia reciente.
- **Branch Target Buffer (BTB):** Almacena direcciones destino de saltos para predecir el próximo PC.
- **Predictor de patrones globales (Gshare):** Usa el historial global de ramas para mejorar precisión.

Los patrones de rama predecibles (e.g., iteraciones de bucle) logran tasas de acierto cercanas al 99 %. Patrones aleatorios o impredecibles pueden degradar el rendimiento severamente.

2.2. Sistemas Multiprocesador y Coherencia de Caché

En arquitecturas multi-núcleo con cachés privadas, el protocolo de coherencia (típicamente MESI o MOESI) garantiza que todos los núcleos tengan una visión consistente de la memoria. Cuando dos núcleos escriben en la misma línea de caché (aunque sea en diferentes bytes), se produce invalidación y tráfico adicional de coherencia.

- **Falso Compartido (False Sharing):** Ocurre cuando dos hilos acceden a variables distintas que residen en la misma línea de caché. El hardware trata estas variables como si fueran compartidas, causando invalidaciones innecesarias y degradación del rendimiento.
- **Solución típica:** Alinear las variables críticas a límites de línea de caché (64 bytes) o usar variables privadas por hilo.

2.3. Eventos de Hardware Relevantes

- **branch-misses:** Número de predicciones erróneas.
- **L1-dcache-load-misses:** Fallos de lectura en caché L1.
- **cache-misses (o LLC-load-misses):** Fallos en el último nivel de caché, indicando tráfico a memoria principal.
- **cycles, instructions:** Permite calcular CPI (Ciclos por instrucción).

3. Requisitos y Configuración del Entorno

3.1. Requisitos de Hardware

- Computador con procesador multi-núcleo (x86-64, mínimo 2 núcleos)
- Sistema operativo Linux (Ubuntu 20.04 o superior)
- 4GB de RAM o más

3.2. Instalación de Herramientas

Listing 1: Instalación de herramientas necesarias

```
1 # Actualizar repositorios
2 sudo apt update
3
4 # Instalar perf y herramientas de desarrollo
5 sudo apt install linux-tools-common linux-tools-generic linux-
6   tools-$(uname -r)
7 sudo apt install g++ make build-essential
8
9 # Instalar soporte para OpenMP (incluido en GCC)
10 # Verificar version: g++ --version
11
12 # (Opcional) Instalar hwloc para ver topologia de cach
13 sudo apt install hwloc
```

3.3. Verificación

```
1 perf --version
2 g++ --version
3 # Verificar n cleos disponibles
4 nproc --all
```

4. Ejercicio 1: Medición del Costo de Fallos de Predicción

4.1. Objetivo

Demostrar cómo la predecibilidad de las ramas afecta el rendimiento y medir la penalización de los fallos de predicción usando `perf`.

4.2. Código Base

Listing 2: branch_prediction.cpp

```
1 #include <iostream>
2 #include <chrono>
3 #include <vector>
4 #include <algorithm>
5 #include <cstdlib>
6 #include <numeric>
7
8 // Funci n que suma elementos seg n umbral, generando una rama
   condicional
9 long long sum_if(const std::vector<int>& data, int threshold) {
10     long long sum = 0;
11     for (int val : data) {
12         if (val > threshold) {
13             sum += val;
14         }
15     }
16     return sum;
17 }
18
19 int main(int argc, char* argv[]) {
20     const size_t N = 100000000; // 100 millones de elementos
21     std::vector<int> data(N);
22
23     // Inicializaci n con patr n predecible: valores ordenados
24     std::iota(data.begin(), data.end(), 0); // 0, 1, 2, ..., N-1
25
26     // Threshold en la mitad -> primera mitad false, segunda
       mitad true
27     int threshold = N / 2;
28 }
```

```

29     std::cout << "Ejecutando con datos predecibles (ordenados)...
        " << std::endl;
30     auto start = std::chrono::high_resolution_clock::now();
31     long long result1 = sum_if(data, threshold);
32     auto end = std::chrono::high_resolution_clock::now();
33     double time_pred = std::chrono::duration<double>(end-start).
        count();
34     std::cout << "Resultado: " << result1 << " - Tiempo: " <<
        time_pred << " s" << std::endl;
35
36     // Mezclar datos aleatoriamente (patr n impredecible)
37     std::random_shuffle(data.begin(), data.end());
38     std::cout << "\nEjecutando con datos aleatorios (impredecible
        )..." << std::endl;
39     start = std::chrono::high_resolution_clock::now();
40     long long result2 = sum_if(data, threshold);
41     end = std::chrono::high_resolution_clock::now();
42     double time_rand = std::chrono::duration<double>(end-start).
        count();
43     std::cout << "Resultado: " << result2 << " - Tiempo: " <<
        time_rand << " s" << std::endl;
44
45     return 0;
46 }

```

4.3. Procedimiento

1. Compilar el programa con optimizaciones desactivadas (para evitar optimización de ramas):

```

1 g++ -O0 -g branch_prediction.cpp -o branch_pred

```

2. Medir eventos de hardware para ambas ejecuciones (por separado, o usando `perf stat` con el programa completo):

```

1 perf stat -e branch-misses,branches,cycles,instructions ./
    branch_pred

```

3. Comparar la tasa de fallos de predicción (`branch-misses / branches`) y el CPI.
4. Opcional: ejecutar con distintos niveles de optimización (`-O1`, `-O2`, `-O3`) y analizar diferencias.

4.4. Preguntas Guía

1. ¿Cuál es la tasa de fallos de predicción para el caso ordenado? ¿Y para el caso aleatorio?
2. ¿Cómo se relaciona el tiempo de ejecución con el número de fallos de predicción?

3. ¿Qué esperaría que ocurriera si se reemplaza `if (val >threshold)` por una operación ternaria o por una suma condicional sin rama (e.g., `sum += val * (val >threshold)`)? ¿Se puede verificar con `perf`?
4. ¿Por qué los procesadores modernos invierten tanta área de silicio en predictores de salto?

5. Ejercicio 2: Análisis de Falso Compartido en Sistemas Multiprocesador

5.1. Objetivo

Detectar y solucionar problemas de falso compartido mediante el uso de `perf` y técnicas de alineación de memoria.

5.2. Código Base: Contador Compartido con Falso Compartido

El siguiente programa incrementa dos contadores globales usando dos hilos OpenMP. Ambos contadores se ubican en la misma línea de caché, generando invalidaciones constantes.

Listing 3: false_sharing.cpp

```
1 #include <iostream>
2 #include <chrono>
3 #include <omp.h>
4
5 // Estructura que provoca falso compartido: ambos enteros en la
6 // misma línea de caché
7 struct Counters {
8     long long counter1;
9     long long counter2;
10 } counters; // global
11
12 void work_on_counter1(int iterations) {
13     for (int i = 0; i < iterations; ++i) {
14         counters.counter1++;
15     }
16 }
17
18 void work_on_counter2(int iterations) {
19     for (int i = 0; i < iterations; ++i) {
20         counters.counter2++;
21     }
22 }
23
24 int main() {
25     const long long ITER = 500000000; // 500 millones de
26     // incrementos por hilo
27
28     double start = omp_get_wtime();
```



```

27     #pragma omp parallel sections
28     {
29         #pragma omp section
30         work_on_counter1(ITER);
31         #pragma omp section
32         work_on_counter2(ITER);
33     }
34     double end = omp_get_wtime();
35
36     std::cout << "Tiempo con falso compartido: " << (end - start)
37               << " s" << std::endl;
38     std::cout << "counter1 = " << counters.counter1 << ",
39               counter2 = " << counters.counter2 << std::endl;
40     return 0;
41 }

```

5.3. Procedimiento

1. Compilar con soporte OpenMP:

```
1 g++ -O2 -fopenmp false_sharing.cpp -o false_sharing
```

2. Ejecutar y medir contadores de caché:

```
1 perf stat -e cache-misses,L1-dcache-load-misses,cycles ./
  false_sharing
```

3. Implementar una versión corregida agregando relleno (padding) para que cada variable esté en líneas de caché distintas:

Listing 4: Solución con padding

```

1 struct AlignedCounters {
2     alignas(64) long long counter1; // forzamos alineación
3     a 64 bytes
4     char padding[64 - sizeof(long long)];
5     alignas(64) long long counter2;
6 } aligned_counters;

```

4. Comparar el rendimiento y las métricas de caché entre ambas versiones.

5.4. Preguntas Guía

1. ¿Cómo afecta el falso compartido al número de `cache-misses` observado?
2. ¿Por qué la solución con `alignas(64)` reduce los fallos? ¿Qué tamaño de línea de caché asume?
3. Si se cambia el número de hilos (e.g., 4 hilos incrementando 4 contadores), ¿cómo empeora el problema?
4. ¿Qué otra técnica (sin padding) podría usarse para eliminar el falso compartido?

6. Ejercicio 3: Optimización de Reducción Paralela y Coherencia

6.1. Objetivo

Aplicar técnicas de privatización y reducción para minimizar el tráfico de coherencia en una operación de suma paralela.

6.2. Código Base: Suma de un Vector

Listing 5: parallel_reduction.cpp

```
1  #include <iostream>
2  #include <vector>
3  #include <chrono>
4  #include <omp.h>
5
6  int main() {
7      const size_t N = 100000000;
8      std::vector<double> vec(N, 1.0);
9      double sum = 0.0;
10
11     double start = omp_get_wtime();
12     #pragma omp parallel for
13     for (size_t i = 0; i < N; ++i) {
14         #pragma omp atomic
15         sum += vec[i];
16     }
17     double end = omp_get_wtime();
18
19     std::cout << "Suma = " << sum << " en " << (end - start) << "
20               s" << std::endl;
21     return 0;
22 }
```

6.3. Procedimiento

1. Compilar y medir con `perf stat` observando el alto costo de la atomicidad.
2. Implementar una versión con reducción de OpenMP:

```
1  double sum = 0.0;
2  #pragma omp parallel for reduction(+:sum)
3  for (size_t i = 0; i < N; ++i) {
4      sum += vec[i];
5  }
```

3. Implementar una versión manual con arreglo privado por hilo y suma final.
4. Comparar tiempos, misses de caché y uso de la barrera de coherencia.

6.4. Preguntas Guía

1. ¿Por qué la versión `atomic` es más lenta que `reduction`?
2. ¿Cómo se comporta el protocolo de coherencia cuando cada hilo escribe en su variable privada?
3. ¿Qué mide el evento `LLC-load-misses` y cómo se relaciona con la escalabilidad?

7. Análisis Comparativo de Técnicas

Cuadro 1: Resumen de técnicas para mitigar penalizaciones en multiprocesador

Técnica	Mecanismo	Evento de hardware reducido
Padding/alineación	Separar variables en distintas líneas de caché	<code>cache-misses</code> , <code>L1-dcache-stores</code>
Privatización	Copias locales por hilo, luego combinación	<code>LLC-load-misses</code> , tráfico de coherencia
Reducción (OpenMP)	Operación asociativa con copias privadas	<code>cache-misses</code> , <code>atomic overhead</code>
Optimización de predicción	Reordenar condiciones para hacerlas predecibles	<code>branch-misses</code> , <code>cycles</code>

8. Entregables

El curso se organiza en grupos de tres y cada grupo deberá entregar un informe en formato PDF con la siguiente estructura:

8.1. Formato del Informe

1. **Portada:** Logo UAI, título, integrantes del grupo, fecha.
2. **Resumen:** Breve descripción de las técnicas evaluadas y resultados principales (máx. 200 palabras).
3. **Introducción:** Conceptos de predicción de saltos y coherencia de caché.
4. **Metodología:** Descripción de experimentos, compilación, comandos `perf` usados.
5. **Resultados:**
 - Tablas con tasas de fallos de predicción y tiempos (Ejercicio 1)
 - Tablas con `cache-misses` antes/después de corregir falso compartido (Ejercicio 2)

- Gráficos comparativos de tiempo de ejecución para diferentes técnicas de reducción (Ejercicio 3)

6. Análisis y Discusión:

- Explicación de por qué los patrones aleatorios empeoran la predicción.
- Impacto del falso compartido en el rendimiento y solución mediante alineación.
- Comparación entre `atomic` y `reduction`.

7. **Conclusiones:** Aprendizajes sobre arquitectura y buenas prácticas de programación paralela.

8. **Código Fuente:** Incluir los archivos `.cpp` modificados en apéndice o enlace a repositorio GitHub.

8.2. Rúbrica de Evaluación

Cuadro 2: Rúbrica del Laboratorio 3

Criterio	Excelente (90-100 %)	Competente (70-89 %)	En Desarrollo (50-69 %)
Correctitud (20 %)	Todo el código compila y ejecuta sin errores	Errores menores, código funcional	Errores significativos
Métricas de predicción (25 %)	Captura detallada de branch-misses, análisis de patrones	Captura básica, análisis superficial	Datos incompletos o erróneos
Métricas multiprocesador (25 %)	Identifica y cuantifica false sharing, mejora ¿50 %	Detecta false sharing pero mejora moderada	No resuelve el problema
Análisis y conclusiones (30 %)	Relación sólida teoría-práctica, recomendaciones concretas	Análisis correcto pero general	Análisis incorrecto o ausente

Normas de Entrega

- El laboratorio debe entregarse una semana después de su publicación (fecha límite: 6 de mayo de 2026).
- Informe en formato PDF, código fuente en repositorio GitHub (enlace en el informe).
- Cada código debe estar en un archivo separado (e.g., `branch_prediction.cpp`, `false_sharing_fixed.cpp`).

- Si se utiliza IA, debe ser referenciada en el informe.
- Retraso: descuento de 0.2 puntos por cada 12 horas. Máximo 5 días de retraso.
- Las notas serán publicadas dentro de 2 semanas posteriores a la entrega; hay 3 días para solicitar corrección.

Apéndice: Resolución de Problemas Comunes

No se encuentran contadores de branch-misses

```

1 # Ejecutar con sudo o ajustar permisos de kernel
2 sudo perf stat -e branch-misses ./program
3 # Ver lista de eventos disponibles
4 perf list | grep branch

```

OpenMP no acelera el código

- Verificar número de hilos: `export OMP_NUM_THREADS=2`
- Compilar con `-fopenmp` y optimizaciones (`-O2`)

El rendimiento empeora con padding

- Asegurar que el tamaño de línea de caché es 64 bytes (común en x86). Usar `getconf LEVEL1_DCACHE_LINESIZE`.
- Probar distintos tamaños de bloque o usar `alignas(std::hardware_destructive_interference_size)` en C++17.

Referencias

1. Hennessy, J. L., & Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann.
2. Intel Corporation. (2024). *Intel VTune Profiler User Guide* (opcional).
3. OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface*.
4. Linux perf Wiki. <https://perf.wiki.kernel.org/>
5. Drepper, U. (2007). *What Every Programmer Should Know About Memory*. Red Hat.